The uNPU from scratch

A 4×4 INT8 weight-stationary systolic matrix-multiply accelerator, explained one piece of hardware at a time

EE5239 · Project 2 · Frankenstein 180 nm IP-Validation SoC
Matches the RTL in `rtl/` (11 modules, APB control port, native valid/ready DMA port, 8-register map)
November 2026

What is inside

1. The problem: one matrix multiply
2. One multiply-accumulate → the processing element
3. From one PE to a 4×4 grid
4. The wavefront problem, and the skew that solves it
5. Getting the results out: de-skew
6. Feeding the array: double-banked buffers
7. Buses from zero: APB and valid/ready
8. The DMA engine
9. The sequencer that runs it all
10. Registers and firmware
11. The whole job, cycle by cycle
12. Where the time goes
13. Traps that cost a day each
14. Appendix: file map, register map, ports

1. The problem: one matrix multiply

Almost every layer of a neural network reduces to one operation:

$$C = A \times B, \qquad C[m]\,[j] = \sum_{k=0}^{K-1} A[m]\,[k] \cdot B[k]\,[j]$$

`A` is `M × K` (activations, say a few input vectors), `B` is `K × N` (weights), `C` is `M × N`. Take the 4×4 example used throughout this document:

```
A = | 1 2 3 4 |      B = | 1 0 2 1 |      C = | 11 13  8  8 |
    | 2 1 0 3 |          | 0 1 1 2 |          |  5  7  8  4 |
    | 0 4 1 1 |          | 2 1 0 1 |          |  3  7  5  9 |
    | 3 0 2 2 |          | 1 2 1 0 |          |  9  6  8  5 |
```

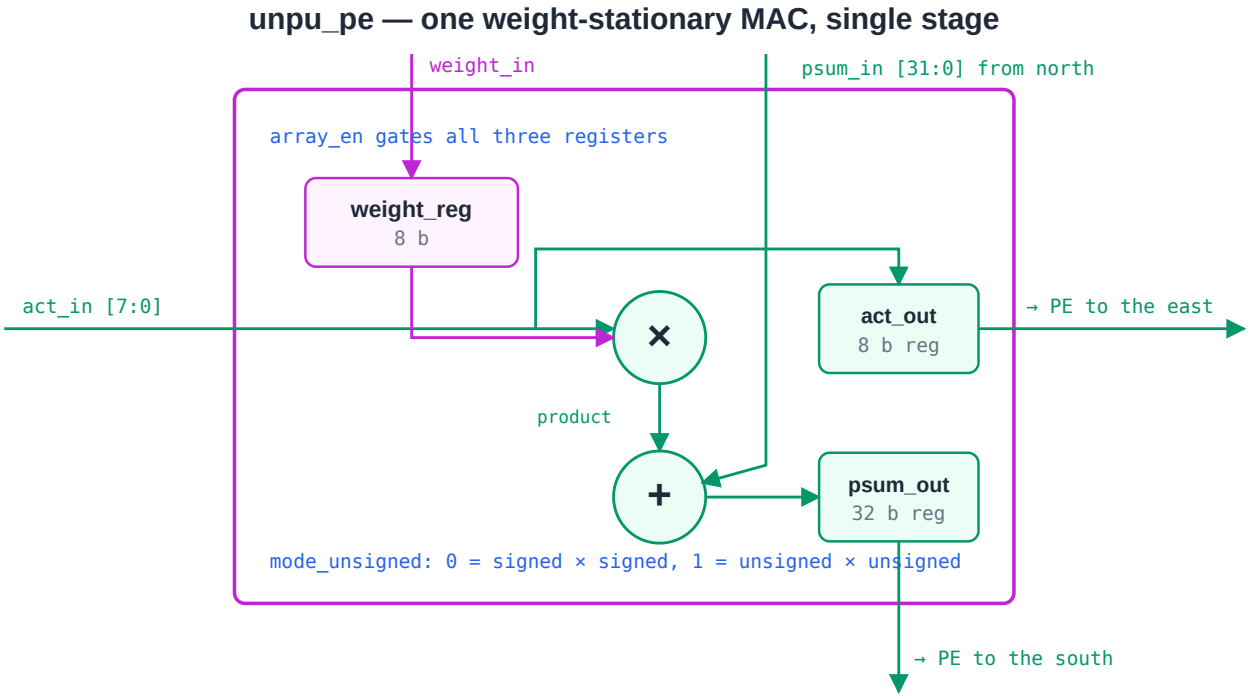
`C[0][0] = 1·1 + 2·0 + 3·2 + 4·1 = 11`. That is 4 multiplies and 3 adds. The whole of `C` needs `M·N·K = 64` multiply-accumulates (MACs).

A RISC-V CPU does those 64 MACs one at a time, plus loads, stores, loop counters and branches: around 500 cycles. The uNPU's job is to do them **16 at a time**, in dedicated hardware, while the CPU does something else.

The uNPU works on INT8 numbers (8-bit integers, the format quantised neural networks use) and produces INT32 results, wide enough that no sum of four INT8 products can overflow (the worst case, `4 × (−128) × (−128) = 65 536`, needs 18 bits).

2. One multiply-accumulate → the processing element

The basic hardware unit is a **MAC**: multiply two numbers, add the product to a running sum. To turn a MAC into a building block for an array we add three registers and fix one operand in place:



The processing element (unpu_pe)

- `weight_reg` holds one weight. It is loaded once per job and then stays put. This is what *weight-stationary* means.
- `act_out` is a one-cycle copy of the activation, passed to the neighbour on the right.
- `psum_out` is `psum_in + weight × act_in`, passed to the neighbour below.

The complete RTL is short enough to read in one go:

```
module unpu_pe (
  input  logic clk, rst_n,
  input  logic array_en, weight_load, mode_unsigned,
  input  logic [7:0]  weight_in, act_in,
  input  logic [31:0] psum_in,
  output logic [7:0]  act_out,
  output logic [31:0] psum_out
);
  logic [7:0]  weight_reg;
  logic [31:0] product;

  always_comb begin
    if (mode_unsigned) product = $unsigned(weight_reg) * $unsigned(act_in);
    else               product = 32'($signed(weight_reg) * $signed(act_in));
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      weight_reg <= '0;  act_out <= '0;  psum_out <= '0;
    end else if (array_en) begin
```

```
            if (weight_load) weight_reg <= weight_in;
            act_out  <= act_in;
            psum_out <= psum_in + product;
        end
    end
endmodule
```

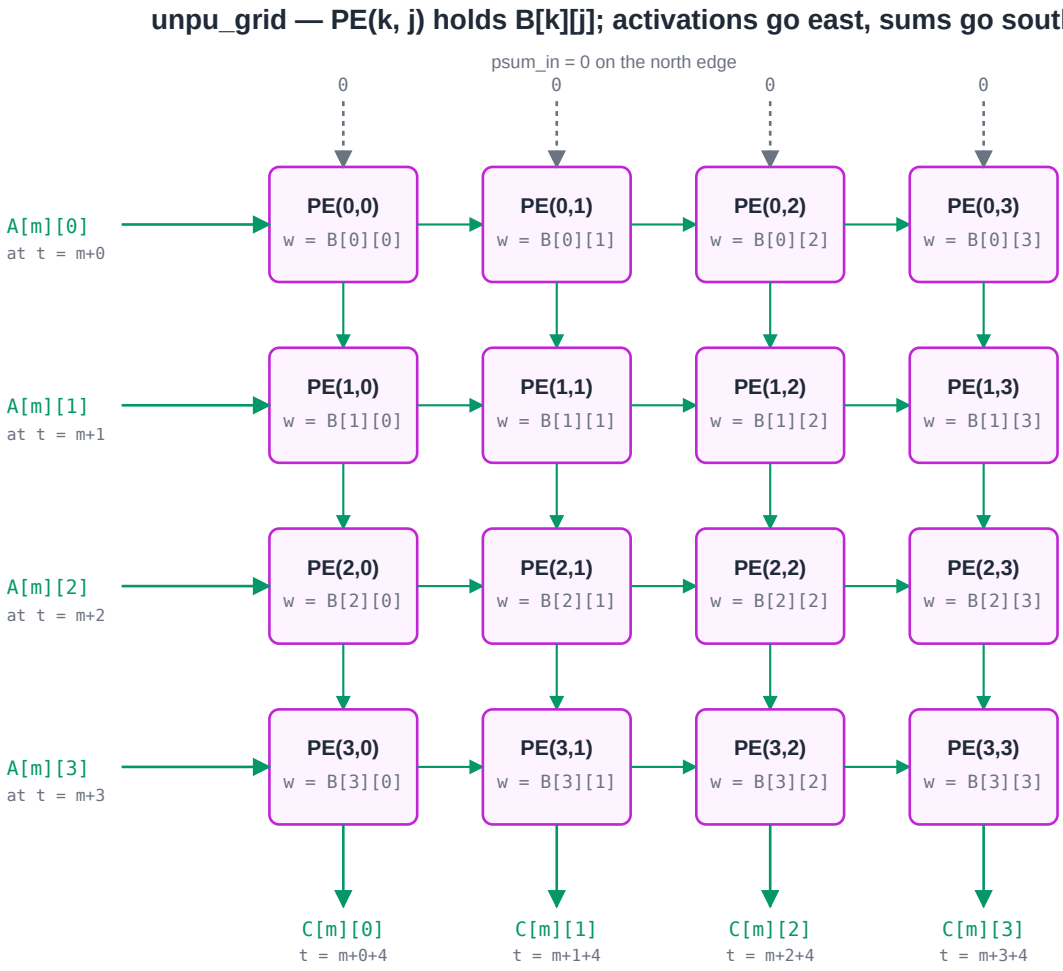
Three design decisions are packed into those lines:

1. **One stage, no pipelining.** The critical path is an 8×8 multiplier followed by a 32-bit adder, about 7 ns in 180 nm. The clock period at 50 MHz is 20 ns, so it fits with room to spare. Pipelining would only be needed above about 130 MHz, and it would change every timing number later in this document.
2. **Signed or unsigned, chosen per job.** `mode_unsigned` selects how the same 8 bits are interpreted. `0xFF` is 255 unsigned but −1 signed.
3. **`array_en` freezes everything.** When it is 0, no register in the PE changes. This is the single mechanism that lets the rest of the chip pause the array, for example while waiting on memory.

Note that `psum_out` is **not** a feedback accumulator: it never reads its own previous value. Each cycle it is completely recomputed from `psum_in` and `product`. That property matters in chapter 9.

3. From one PE to a 4×4 grid

Put 16 PEs in a 4×4 grid. Connect each PE's `act_out` to the `act_in` of the PE on its right, and each `psum_out` to the `psum_in` of the PE below. Load **PE(k, j)** with `B[k][j]`:



The 4×4 grid (unpu_grid)

Now look at column `j` alone. If the activation `A[m][k]` arrives at row `k` at the right moment, row 0 computes `A[m][0]·B[0][j]`, row 1 adds `A[m][1]·B[1][j]`, row 2 adds the third term, row 3 the fourth. What leaves the bottom of column `j` is exactly `C[m][j]`. The four columns hold different weights, so they produce the four elements of row `C[m]` in parallel. Every row of A that streams in produces one row of C.

The RTL is two `generate` loops that pick each PE's inputs:

```
for (row = 0; row < 4; row++) begin : g_row
  for (col = 0; col < 4; col++) begin : g_col
    if (col == 0) assign pe_act_in  = act_in[row];          // west edge
    else          assign pe_act_in  = act_link[row][col-1]; // from the left
    if (row == 0) assign pe_psum_in = psum_in[col];         // north edge (tied 0)
    else          assign pe_psum_in = psum_link[row-1][col];// from above
    unpu_pe pe (.act_in(pe_act_in), .psum_in(pe_psum_in),
                .act_out(act_link[row][col]), .psum_out(psum_link[row][col]), ...);
```

```
        end
      end
```

Nothing in the grid is long or shared: every wire goes to an immediate neighbour, which keeps routing short and timing easy, the whole point of a systolic design.

4. The wavefront problem, and the skew that solves it

The partial sum for row `m` leaves PE(0, j) after one cycle, reaches row 1 a cycle later, row 2 a cycle after that. So **row `k` must receive `A[m][k]` exactly `k` cycles after row 0 received `A[m][0]`**. Activations also travel one column per cycle to the right, so PE(k, j) sees `A[m][k]` at cycle `m + k + j`.

Here is column 0 of the example, showing the partial sum held in each PE's `psum_out` at the end of each cycle (`-` = not yet relevant):

End of cycle	PE(0,0)	PE(1,0)	PE(2,0)	PE(3,0)
0	1 (m=0)	–	–	–
1	2 (m=1)	1	–	–
2	0 (m=2)	2	7	–
3	3 (m=3)	0	2	**11** = C[0][0]
4	–	3	2	**5** = C[1][0]
5	–	–	7	**3** = C[2][0]
6	–	–	–	**9** = C[3][0]

Read it diagonally: the value for `m = 0` moves down one row per cycle, gaining one term at each step: `1 → 1 → 7 → 11`. This diagonal is the **wavefront**.

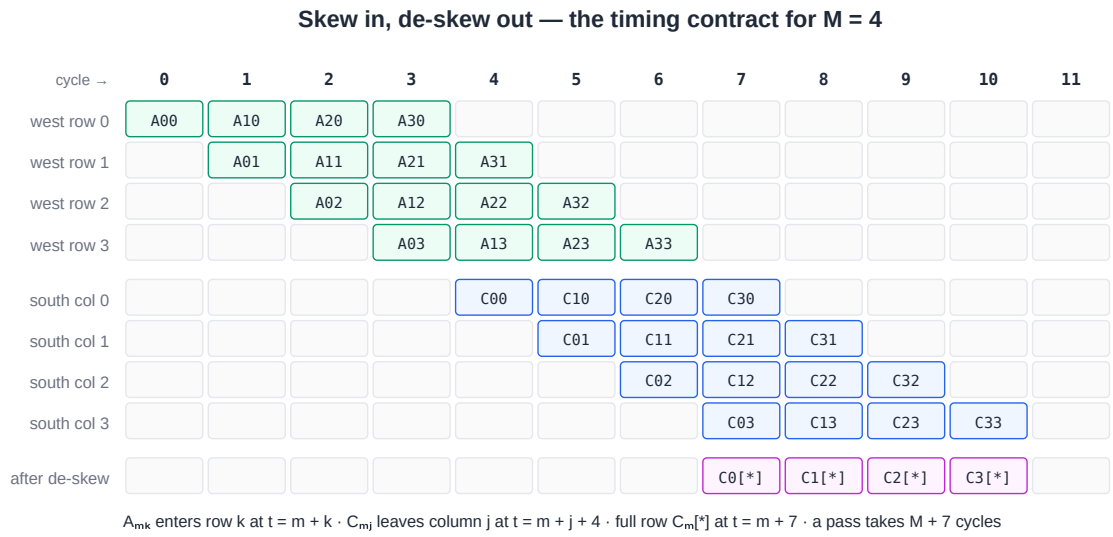
The buffer, though, hands over a whole row `A[m][0..3]` at once. Something must delay element `k` by `k` cycles. That is `unpu_skew`: row 0 is a plain wire, row 1 one flop, row 2 two, row 3 three.

```
assign act_out[0] = a_raw[0];                    // depth 0: a wire, never a flop
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) ...
  else if (array_en) row1_q <= a_raw[1];         // depth 1
assign act_out[1] = row1_q;
// rows 2 and 3: two and three flops in a chain
```

The depths are written out by hand rather than with a parameterised `generate`, so nobody can accidentally put a flop on row 0.

5. Getting the results out: de-skew

The same stagger appears at the bottom. Column `j`'s result for row `m` leaves the grid at cycle `m + j + 4` (the 4 is the four rows it fell through). Column 0 is earliest, column 3 latest. `unpu_deskew` mirrors the skew bank: column 0 gets three flops, column 3 none, so all four columns of `C[m]` arrive together at cycle `m + 7`.



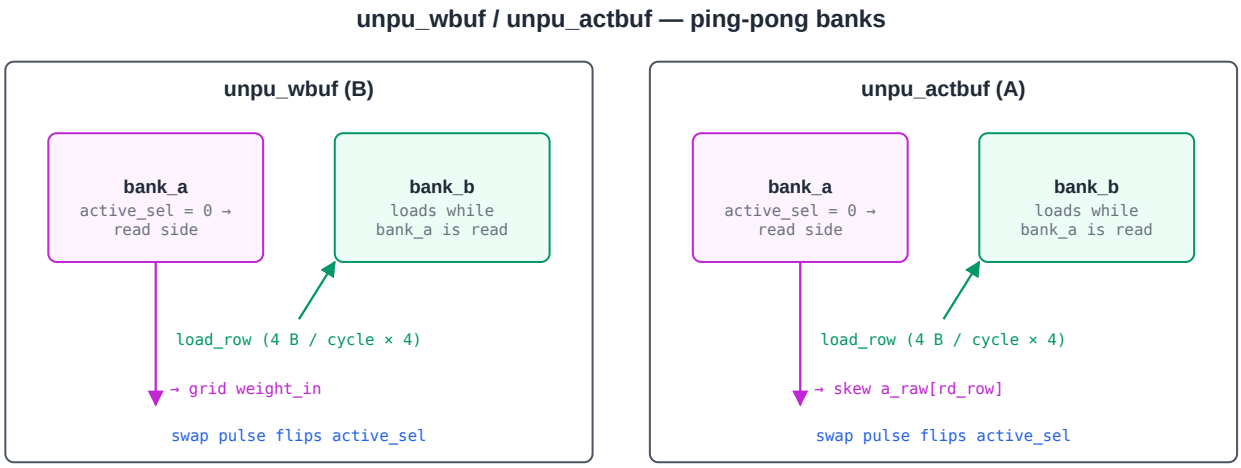
Skew in, de-skew out

These five equations are the **timing contract**. Every module in the design agrees with them, and changing any one of them is a design-wide change:

```
A[m][k] enters west edge of row k      at cycle  m + k
A[m][k] reaches PE(k, j)               at cycle  m + k + j
C[m][j] leaves south edge of column j  at cycle  m + j + 4
whole row C[m][*] valid after de-skew  at cycle  m + 7
one pass of M rows takes                         M + 7 cycles
```

6. Feeding the array: double-banked buffers

The array needs 16 weights at once and one row of activations per cycle, but memory delivers one 32-bit word (four INT8 values) at a time, and only when the arbiter allows it. Two small buffers sit in between.



wbuf: rows shifted in bottom-up (row 3 first), masked to K×N, transposed; all 16 PEs load in parallel on w_swap.
actbuf: rows written by index (row 0 first), masked to M×K; read combinationally by rd_row = COMPUTE cycle.

Ping-pong banks

Each buffer has **two banks**. One is *active* and feeds the datapath; the other *loads* from the DMA. A one-cycle `swap` pulse exchanges their roles. With one job at a time, the second bank mainly guarantees that a load can never disturb data the array is currently using; it is also the hook for overlapping the next job's fetch later.

Both buffers load in exactly four cycles, one row per cycle, and **mask** anything outside the real matrix size to zero. That is how a 2×3 job runs on 4×4 hardware: the extra rows and columns simply contain zeros and add nothing to the sums.

```
        // unpu_wbuf: zero any row >= K and any column >= N
        assign masked_row[c] = (row_idx >= eff_k) ? 8'h00 :
                               ((c >= eff_n)      ? 8'h00 : load_row[c]);
```

The weight buffer (`unpu_wbuf`) shifts rows in from the bottom, row 3 first, and transposes on the way out so that `weight_in[r][c]` is `B[r][c]`. On the `w_swap` cycle it raises all 16 `weight_load` bits together and shows the *new* bank, so the whole 4×4 weight matrix lands in the PEs in a **single cycle**:

```
        assign weight_load     = swap ? '1 : '0;
        assign effective_sel   = active_sel ^ (swap && array_en);
        assign weight_in[r][c] = active_bank[c][r];
```

The activation buffer (`unpu_actbuf`) stores rows by index and reads them combinationally. The sequencer drives `rd_row`, and row `rd_row` appears on `rd_data` in the same cycle, with no clock in between:

```
        bank_b[row_idx[1:0]] <= masked_row;   // load the inactive bank
        assign rd_data = active_bank[rd_row]; // zero-latency read into the skew bank
```

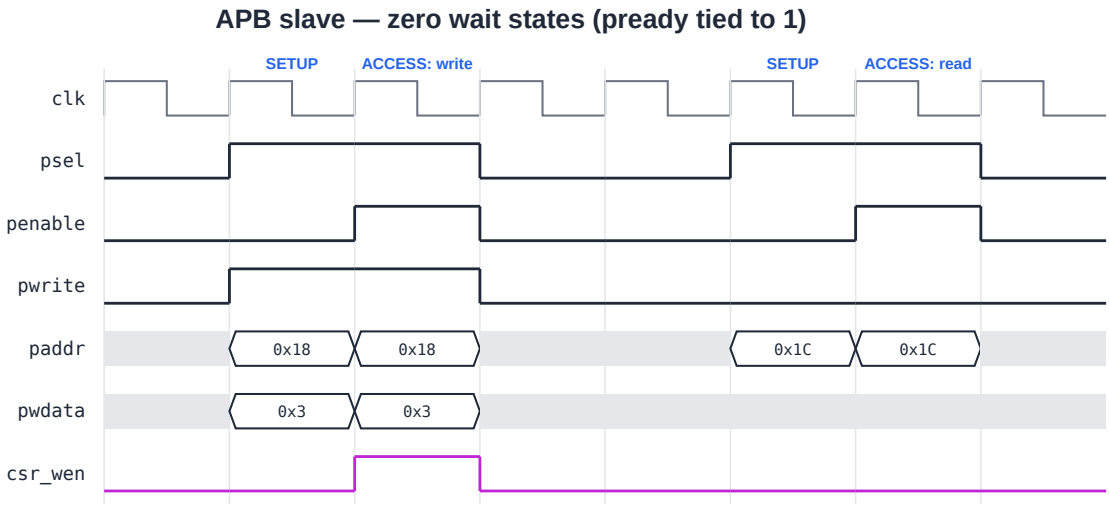
7. Buses from zero: APB and valid/ready

A bus is an agreement about which wires mean what, and on which clock edge. The uNPU speaks two.

7.1 APB, the control port

APB (ARM's Advanced Peripheral Bus) is the simplest bus in common use. A **master** (the CPU's APB bridge) talks to a **slave** (the uNPU) in two phases:

- **SETUP:** the master raises `psel` (this slave is selected), puts the address on `paddr`, sets `pwrite` (1 = write) and, for writes, the data on `pwdata`. `penable` is 0.
- **ACCESS:** the next cycle the master raises `penable`. The transfer completes at the end of any ACCESS cycle in which the slave's `pready` is 1. Read data is returned on `prdata`.



APB: a write to npu_ctrl, then a read of npu_status

The uNPU's registers can always be read or written in one cycle, so `pready` is tied to 1 and the whole APB slave is five lines with no flip-flops:

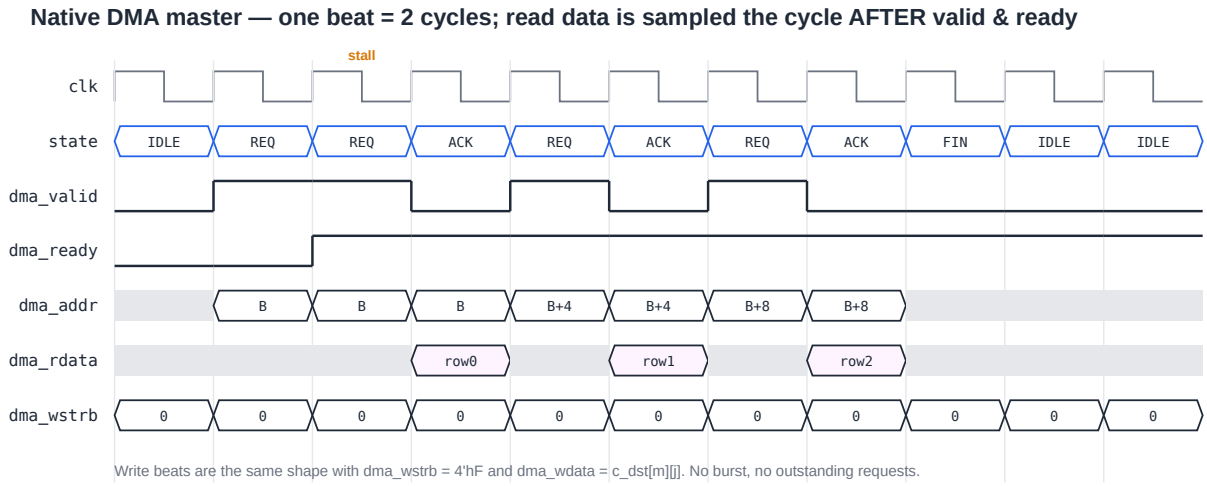
```
assign pready    = 1'b1;
assign csr_sel   = paddr[11:2];              // word index in the 4 KB window
assign csr_wdata = pwdata;
assign csr_wen   = psel && penable && pwrite; // only in ACCESS
assign prdata    = csr_rdata;
```

The write strobe uses `penable` on purpose: in SETUP the master may still be settling the address and data.

7.2 Valid/ready, the data port

The data port is a simpler native handshake to the SRAM arbiter. The uNPU is the **master** here. Several masters (CPU instruction fetch, CPU data, uNPU DMA, SPI) compete for one SRAM, so the arbiter must be able to say "not yet":

- The master raises **`dma_valid`** with `dma_addr`, `dma_wstrb` (`4'hF` = write, `4'h0` = read) and, for writes, `dma_wdata`. It holds all of them steady.
- The arbiter raises **`dma_ready`** in the cycle it accepts. The transfer happens on that clock edge.
- For a read, **`dma_rdata` is valid in the following cycle**.



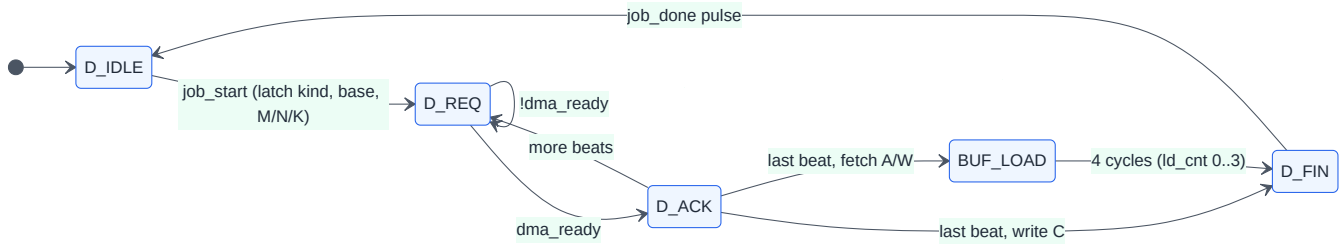
Three reads; the first one is stalled for a cycle

One request is outstanding at a time and there are no bursts, so every beat costs at least two cycles. The arbiter can stretch any beat as long as it wants.

8. The DMA engine

`unpu_dma` is a small state machine that performs one of three **jobs**, each a sequence of single-word beats:

job_kind	Job	Beats	Address of beat	Destination
0	FETCH_A	M	`src_a + 4·beat`	`unpu_actbuf`
1	FETCH_W	K	`src_b + 4·beat`	`unpu_wbuf`
2	WRITE_C	M·N	`dest_c + 16·m + 4·j`	SRAM



DMA state machine

A fetch collects its words in a 4-entry `stage` register file, then spends four `BUF_LOAD` cycles streaming them into the right buffer (in reverse order for the weight buffer's bottom-up shift). A write takes its data straight from the sequencer's captured result matrix.

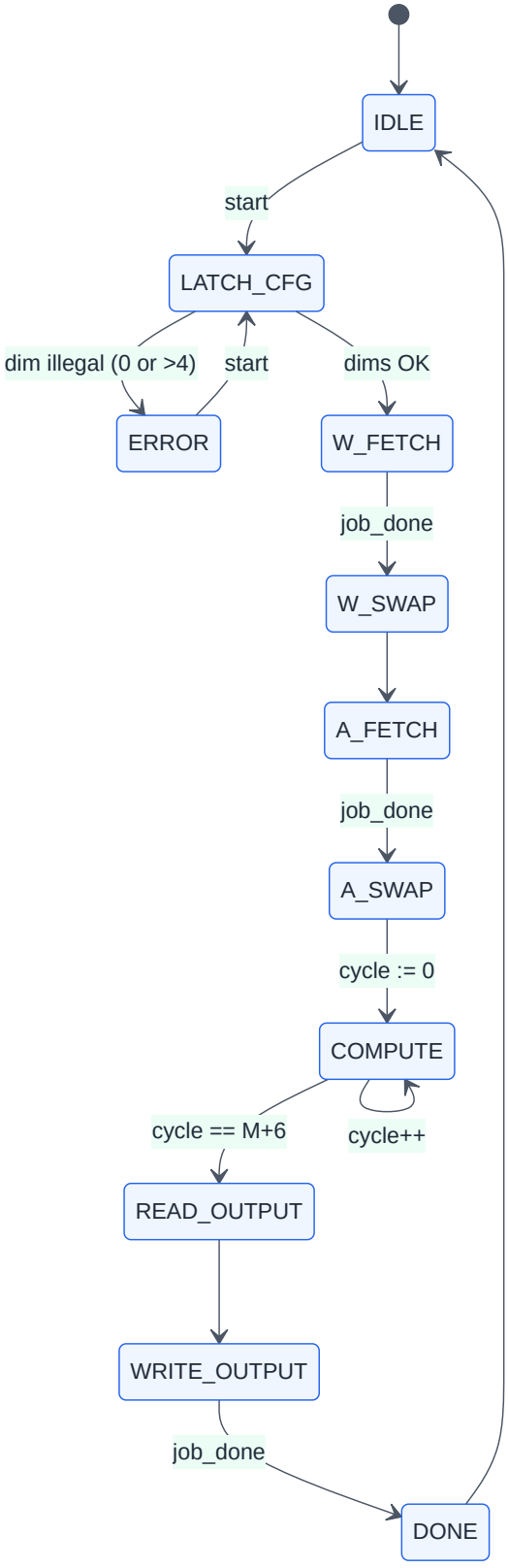
```
        assign dma_valid = (state == D_REQ);
        assign dma_wstrb = (state == D_REQ && kind_lat == JOB_WRITE_C) ? 4'hF : 4'h0;
        assign dma_addr  = (kind_lat == JOB_WRITE_C)
                         ? base_lat + cur_m*16 + cur_j*4
                         : base_lat + beat_idx*4;

        D_REQ: if (dma_ready) state <= D_ACK;          // wait as long as needed
        D_ACK: begin
          if (kind_lat != JOB_WRITE_C) stage[beat_idx[1:0]] <= dma_rdata;
          if (beat_idx == total_beats - 1) state <= (kind_lat == JOB_WRITE_C) ? D_FIN :
BUF_LOAD;
          else begin beat_idx <= beat_idx + 1; state <= D_REQ; end
        end
```

The address only advances in `D_ACK`, after the handshake, never on `valid` alone. That is the most common DMA bug: it passes with a memory that is always ready and fails the moment the arbiter stalls.

9. The sequencer that runs it all

`unpu_seq` turns one START into the whole job. It never touches data itself. It dispatches DMA jobs, pulses the buffer swaps, counts compute cycles and captures results.



Sequencer state machine

LATCH_CFG checks that every dimension is between 1 and 4 (otherwise **ERROR** with `error_code = 1`) and copies dimensions, mode and pointers into shadow registers, so firmware changing a register mid-job cannot corrupt it.

W_FETCH → W_SWAP → A_FETCH → A_SWAP brings the operands in. Each dispatch state fires `job_start` exactly once, guarded by a `job_issued` flag, and then waits for `job_done`, however long the arbiter takes:

```
assign job_start = is_dispatch_state && !job_issued;   // once per visit
W_FETCH: if (job_issued && job_done) state_n = W_SWAP;
```

COMPUTE runs for `M + 7` cycles with a counter `cycle`. During the first `M` cycles it selects activation row `cycle` (combinationally, so there is no lag), and from cycle 7 it captures one de-skewed result row per cycle:

```
assign rd_row = (state == COMPUTE && cycle < m_lat) ? cycle[1:0] : 2'd0;
COMPUTE: if (cycle == m_lat + 6) state_n = READ_OUTPUT;
...
else if (state == COMPUTE && cycle >= 7) c_dst[cycle - 7] <= c_in;
```

WRITE_OUTPUT dispatches the write-back job, and **DONE** produces a one-cycle `done` pulse.

`array_en` is held at 1 from W_FETCH all the way through WRITE_OUTPUT, including the possibly long DMA waits. That is safe because of the property noted in chapter 2: no PE remembers anything beyond one register stage, and capture only starts at cycle 7, when everything that flowed through earlier has been flushed by real data.

All of the sequencer's outputs are **Moore** outputs, functions of the current state only, which keeps their timing obvious.

10. Registers and firmware

`unpu_csr` holds eight 32-bit registers in the 4 KB window starting at `0x4000_0000`:

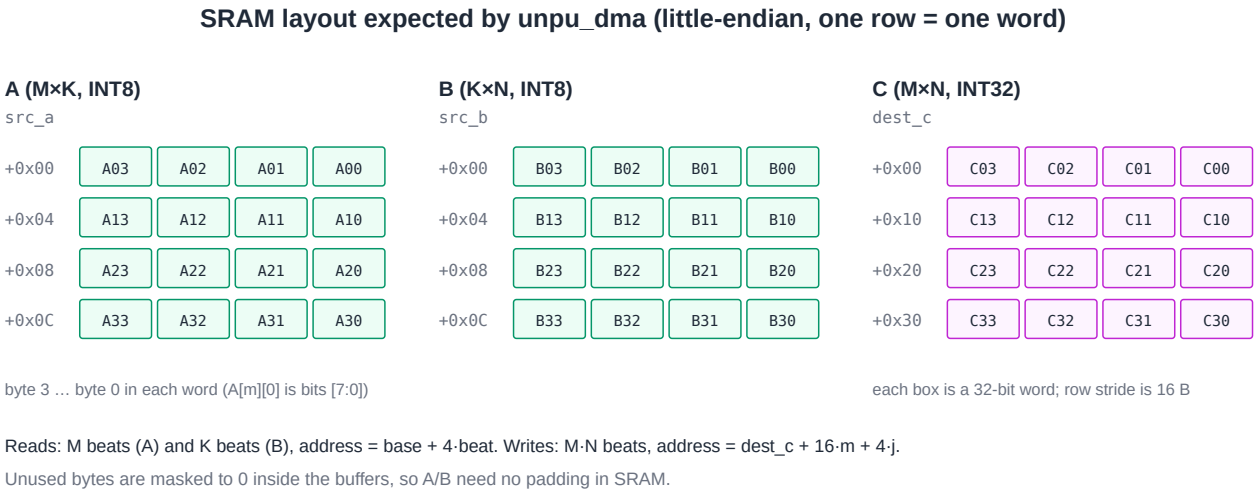
Offset	Name	Access	Meaning
`0x00`	`src_a`	R/W	SRAM address of A
`0x04`	`src_b`	R/W	SRAM address of B
`0x08`	`dest_c`	R/W	SRAM address of C
`0x0C`	`dim_m`	R/W	M, 1–4
`0x10`	`dim_n`	R/W	N, 1–4
`0x14`	`dim_k`	R/W	K, 1–4
`0x18`	`npu_ctrl`	R/W	bit 0 START (write 1 to pulse, reads 0) · bit 1 SIGNED
`0x1C`	`npu_status`	RO	bit 0 DONE (sticky) · bit 1 ERROR · bits 4:2 error code

Two behaviours deserve a closer look:

```
        // DONE stays set until the next START, so firmware can poll whenever it likes
        if      (start_pulse) status_done_reg <= 1'b0;
        else if (done_i)      status_done_reg <= 1'b1;

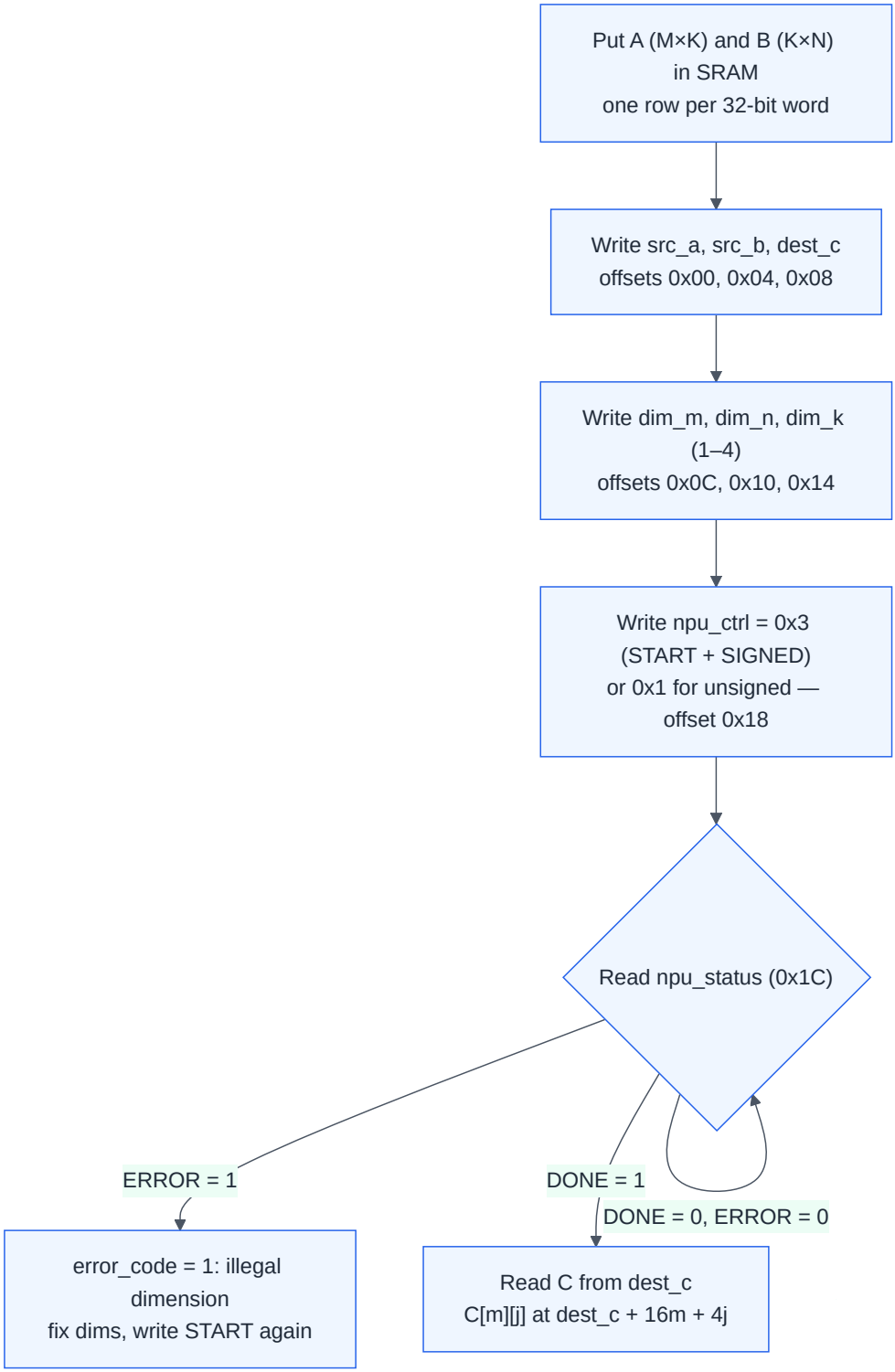
        // The register bit means SIGNED; the PE port means UNSIGNED
        assign mode_unsigned = ~ctrl_signed_reg;
```

The SIGNED bit resets to 0, so the default is unsigned. Matrices are stored in SRAM one row per 32-bit word, little-endian; results are INT32 with a fixed 16-byte row stride:



SRAM layout

The firmware sequence is short:



Firmware flow

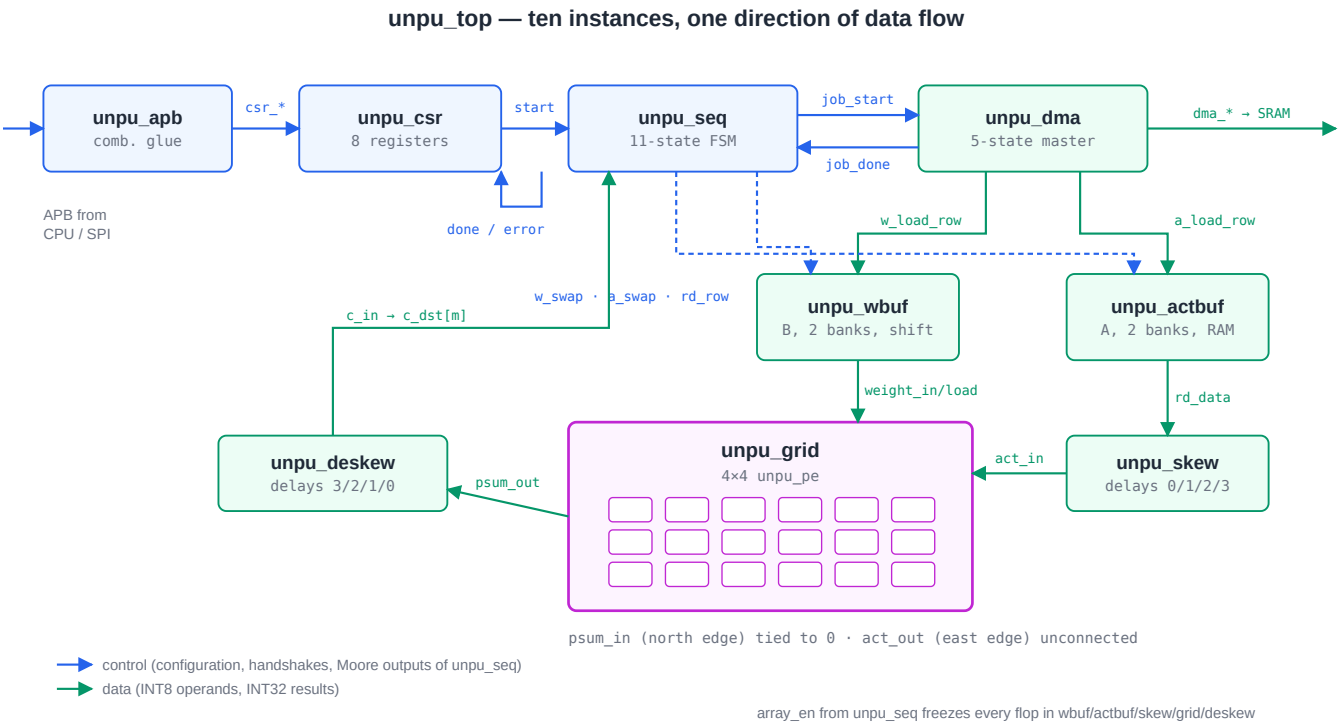
```
#define NPU(off) (*(volatile uint32_t *)(0x40000000u + (off)))

NPU(0x00) = 0x1200;    NPU(0x04) = 0x1000;    NPU(0x08) = 0x1400;  /* A, B, C   */
NPU(0x0C) = 4;         NPU(0x10) = 4;         NPU(0x14) = 4;       /* M, N, K   */
```

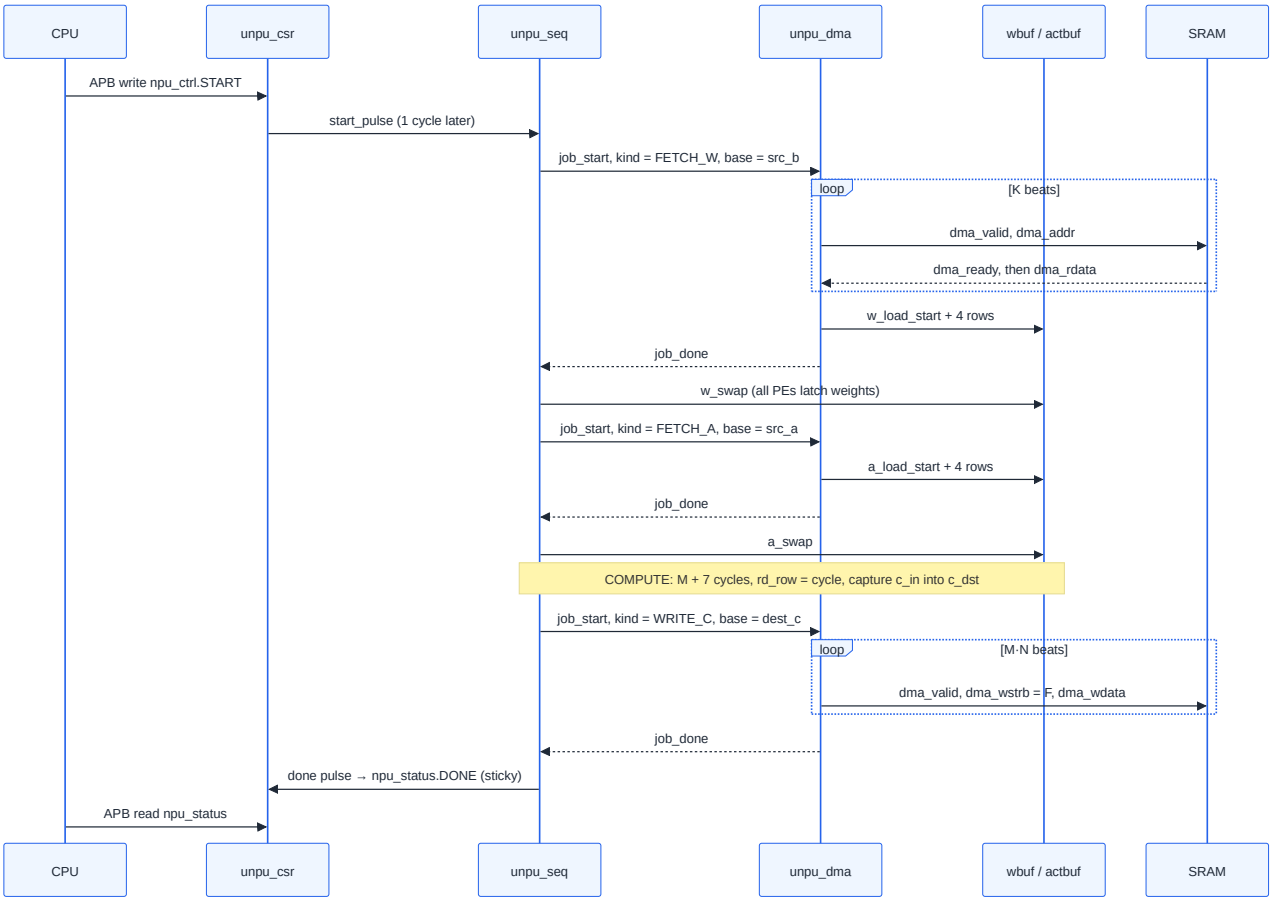
```
        NPU(0x18) = 0x3;                                        /* SIGNED+GO */
        while ((NPU(0x1C) & 0x3) == 0) ;                        /* poll      */
```

11. The whole job, cycle by cycle

Putting every block together:



The ten instances of unpu_top

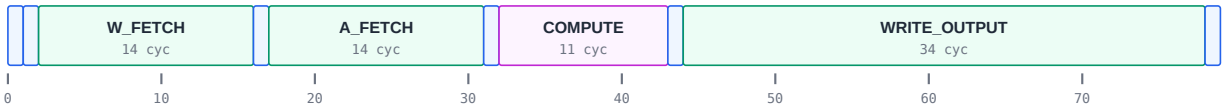


Every handshake in one job

For the 4×4×4 example with a memory that is always ready, a cycle model of the sequencer and DMA gives these state spans (cycle 0 is the cycle `start_pulse` is high):

Cycles	State	What is happening
0	IDLE	`start_pulse` seen
1	LATCH_CFG	dims checked, config latched
2–15	W_FETCH	4 reads of B (8 cycles), 4 cycles shifting into `unpu_wbuf`, `job_done`
16	W_SWAP	all 16 weights load into the PEs
17–30	A_FETCH	4 reads of A, 4 cycles into `unpu_actbuf`
31	A_SWAP	activation bank becomes active, `cycle = 0`
32–42	COMPUTE	rows 0–3 injected at cycles 0–3, results captured at cycles 7–10
43	READ_OUTPUT	—
44–77	WRITE_OUTPUT	16 INT32 writes, 2 cycles each
78	DONE	`done` pulse; `npu_status.DONE` set next cycle

One 4×4×4 job, ideal memory (dma_ready = 1): 78 cycles from start_pulse to DONE



`W_FETCH = 1 + 2K + 4 (buffer load) + 1 · A_FETCH = 1 + 2M + 4 + 1 · COMPUTE = M + 7 · WRITE_OUTPUT = 1 + 2·M·N + 1`

Total: cycles(start_pulse → DONE) = 26 + 2K + 3M + 2MN

Write-back of 16 INT32 words is 44 % of the job: the 32-bit memory port, not the array, sets the speed.

Job timeline

In general:

$$\text{cycles}\left(\text{ start_pulse} \rightarrow \text{DONE}\right) = 26 + 2K + 3M + 2MN$$

The same model produces the correct C for random signed matrices of every size from 1×1×1 to 4×4×4, and still produces it when `dma_ready` is low two cycles in three (the job then takes 99 cycles instead of 78).

12. Where the time goes

M × N × K	Cycles	Ops (2MNK)	At 50 MHz
1 × 1 × 1	33	2	3 MOPS
2 × 2 × 2	44	16	18 MOPS
4 × 4 × 1	72	32	22 MOPS
4 × 4 × 4	78	128	82 MOPS

The array's peak is 16 MACs per cycle, 1.6 GOPS at 50 MHz. A full job reaches about 5 % of that. The array is busy for only 11 of 78 cycles. The rest is the **single 32-bit memory port**: 8 cycles to read A, 8 to read B, and 32 to write C.

That is the most useful thing to know about this design. Making the array faster changes almost nothing. The improvements that matter, roughly in order of payoff:

1. **Pack the output to INT8** (requantise in hardware): write-back drops from 32 cycles to 8.
2. **Overlap phases**: the double banks would let the next job's A or B be fetched while the current job computes (the sequencer does not do this yet).
3. **Accumulate across K tiles** in hardware (use the north-edge `psum_in`) so larger matrices do not round-trip partial sums through SRAM.

13. Traps that cost a day each

Trap	Symptom	Where it is handled
A flop on a depth-0 lane	Every result shifted by one row	`unpu_skew` row 0, `unpu_deskew` col 3 written as wires
`rd_row` registered from `cycle`	Row m arrives a cycle late	`assign rd_row = ...` in `unpu_seq`
`job_start` held for the whole wait	DMA restarts every cycle	`job_issued` flag
Address advanced on `valid`	Passes at 100 % ready, fails under stalls	address moves only in `D_ACK`
Read data taken in the handshake cycle	Every word shifted by one beat	`dma_rdata` used in `D_ACK`
SIGNED vs unsigned polarity	Silent wrong answers for negative numbers	one inversion in `unpu_csr`
Weights loaded top-first	`A × flipud(B)`, plausible and wrong	reverse order in `unpu_dma` + bottom-up shift in `unpu_wbuf`
DONE not cleared by START	Second job “finishes” instantly	start-clear priority in `unpu_csr`
No input skew	A row of zeros in C	`unpu_skew`

14. Appendix

Files

File	Lines	Contents
`unpu_pe.sv`	48	processing element
`unpu_grid.sv`	70	4×4 array
`unpu_skew.sv`	73	input delays 0/1/2/3
`unpu_deskew.sv`	71	output delays 3/2/1/0
`unpu_wbuf.sv`	218	weight buffer
`unpu_actbuf.sv`	155	activation buffer
`unpu_dma.sv`	234	bus master
`unpu_seq.sv`	338	main sequencer
`unpu_csr.sv`	170	registers
`unpu_apb.sv`	93	APB slave
`unpu_top.sv`	286	top-level wiring

Macro ports (`unpu_top`)

Group	Signals	Pins
Global	`clk`, `rst_n`	2
APB slave	`paddr[31:0]`, `pwdata[31:0]`, `pwrite`, `psel`, `penable` in; `prdata[31:0]`, `pready` out	100
DMA master	`dma_addr[31:0]`, `dma_wdata[31:0]`, `dma_wstrb[3:0]`, `dma_valid` out; `dma_rdata[31:0]`, `dma_ready` in	102

Current limits

One tile of at most 4×4×4 per job; no hardware accumulation across tiles; INT32 output only; status by polling (no interrupt pin); no DMA timeout; register offsets are a proposal pending PM confirmation.